



Aras Innovator 33

MAC Policies

Document #: D-008114

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	4
1 Overview	5
1.1 MAC Policy Concepts	5
1.1.1 How MAC Works	6
1.1.2 MAC Policy Rules	6
1.1.3 MAC Conditions	7
1.1.4 Boolean Expressions	7
1.1.5 MAC Attributes	9
1.1.6 Enabling Properties for Use in MAC Expressions	9
1.1.7 Environment Attributes	10
1.1.8 Derived Multivalued Attributes	10
2 Establishing MAC Policies	12
2.1 Creating a New MAC Policy	12
2.1.1 Creating a MAC Condition	14
2.1.2 Using xClasses and xProperties in MAC Policy Conditions	17
2.2 Defining Environment Attributes	18
2.3 Derived (Multi-Valued) Attributes	20
2.3.1 Configuring a Derived Attribute	20
2.4 Applying Policy Rules	22
2.4.1 Applying MAC Policies	23
2.4.2 Updating MAC Policies	24
2.5 Exempt Identities	24
3 Appendix	25
3.1 Sample Environment Attribute Method	25
3.2 Sample MAC Policy Rule	25

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation

100 Brickstone Square

Suite 100

Andover, MA 01810

Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

1 Overview

Aras Innovator provides the flexibility to give administrators many options when controlling permissions and access. Mandatory Access Control (MAC) is designed to provide access control to Items based on properties of the Item being requested, and properties of the user requesting the Item. MAC Policies can be created for several different use cases where a user's properties (Clearance Level, Citizenship, Location, etc..) determine access rights to Items which may also have various restricting properties (Classification, Ownership, Cost, etc.).

In addition to properties directly on Itemtypes and users, attributes can be derived from related items and used in conditional logic for the access control policy. For example, a policy may want to derive a list of security level restrictions from all referencing Projects for a Part.

Custom logic can also be used to determine environmental conditions, like time of day or other customer-specific requirements. All of these features empower administrators to implement a flexible and robust security scheme.

This overview provides a conceptual understanding of MAC Policy. Section 2 will guide you through the process of creating MAC Policies with step-by-step instructions.

1.1 MAC Policy Concepts

Mandatory Access Control (MAC) imposes an additional layer of access control over Item permissions and Domain Access Control (DAC). MAC does not grant any access—it simply revokes or allows existing access by enforcement of a final and overriding set of conditions for access control using (a) the attributes of the item being accessed as well as (b) attributes of the User who is requesting access dynamically at the time the request is made.

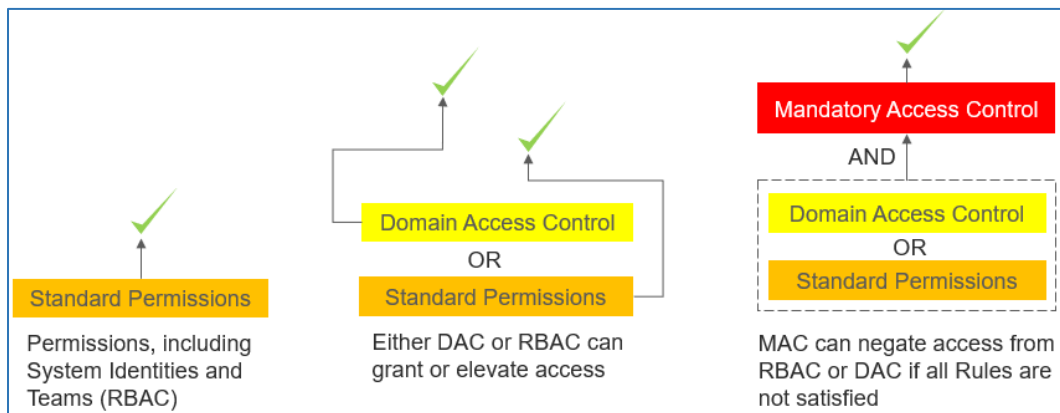


Figure 1.

1.1.1 How MAC Works

In its most basic usage MAC utilizes property values from the Item being accessed and the User requesting access and uses them in Condition logic to control access. Conditions are applied to Access Rights ('Get', 'Update', 'Delete', etc) to define a MAC Policy Rule. A set of MAC Policy Rules defines a MAC Policy, which can be applied to specified Itemtypes, e.g., *Document* in the example below.

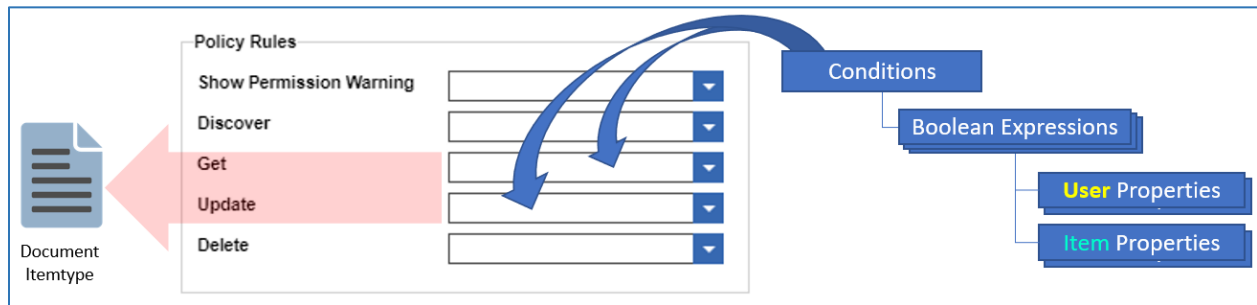


Figure 2.

For example, the hypothetical Condition Logic for the above MAC controlled Document might contain the following boolean expressions:

1. Current **Item** (E.g., Document) has a property "requires security" set to TRUE
2. AND: The current **User** has a property "security clearance" with a value greater than 0
3. AND: The current **User** property "foreign national" is set to FALSE
4. AND: The Environment Attribute *Within_Accessible_Hours* is TRUE

If the result of the overall Condition is TRUE then the access rights it is associated with ('Get', 'Update') will remain unaffected for the requesting user. If the result of the Condition is NOT true, then MAC will unilaterally revoke any existing "Get" and "Update" access for the current user. In this manner, the MAC Policy effectively provides a final exclusionary filter over all other Access Controls that may be in place via RBAC and/or DAC.

1.1.2 MAC Policy Rules

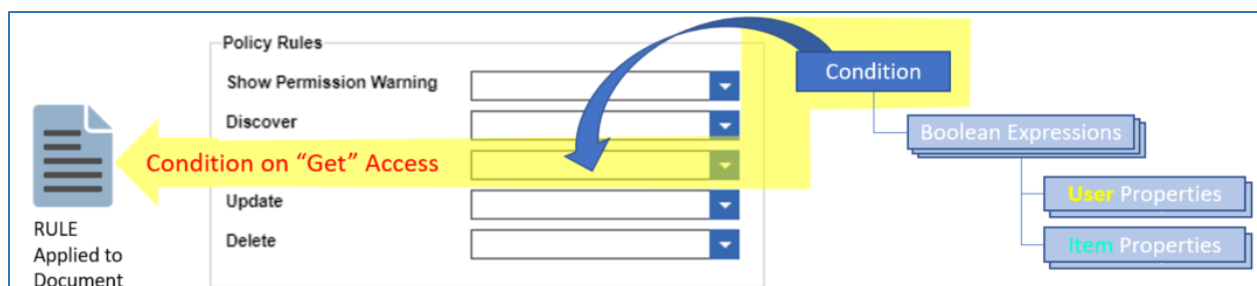


Figure 3.

When a Condition is applied to an Access Right ("Get" in this example), then it defines a *MAC Policy Rule* for the Itemtype(s) that the MAC Policy applies to (Document in this example).

Configuring a Rule is a simple selection of an existing Condition item in the access type row. The actual logic behind the Rule is defined in the **Condition** Editor, and the Condition is defined in terms of the boolean expressions contained therein

1.1.3 MAC Conditions

Before getting into the step-by-step details of establishing a MAC Policy in [Section 2](#) it's helpful to understand the hierarchy of Attributes, Boolean Expressions, and Operators that comprise a MAC Policy Condition.

Logically, the hierarchy looks like shown in Figure 4.

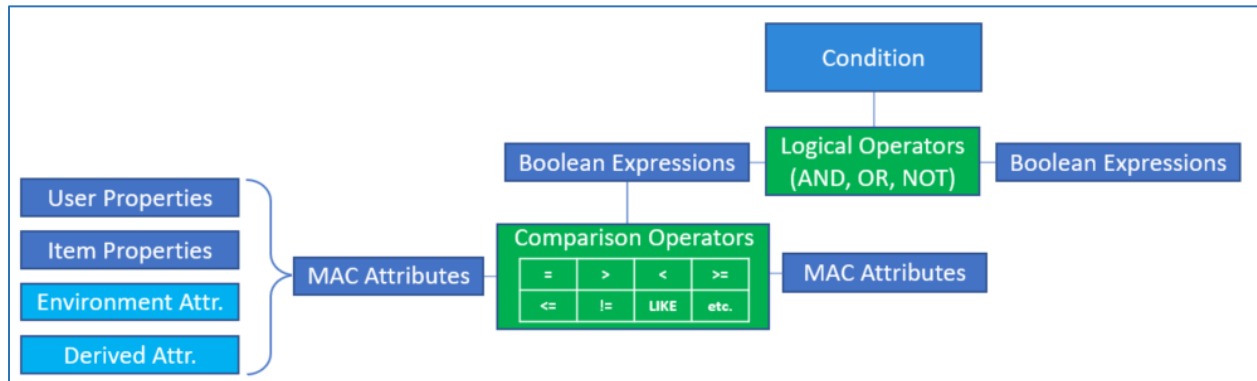


Figure 4.

Boolean Expressions can be combined by Logical Operators AND, OR, NOT ultimately producing one TRUE or FALSE result as the Condition (parent). Recall the simple example from the Introduction [Section 1.1](#):

- Current **Item** (E.g., Document) has a property “requires security” set to TRUE
- AND: The current **User** has a property “security clearance” with a value greater than 0
- AND: The current **User** property “foreign national” is set to FALSE
- AND: The Environment Attribute *Within_Accessible_Hours* is TRUE

The result of evaluating all of these Boolean expressions as combined with Logical Operators is either TRUE or FALSE, therefore we can think of the Condition itself as having a boolean result for the Rule where it would be applied.

1.1.4 Boolean Expressions

Boolean expressions are the unit expressions that produce a TRUE/FALSE result using CurrentItem and CurrentUser objects. They are combined with Logical Operators (AND, OR NOT) and used to define Conditions.



Figure 5.

For example, the above contains:

- Boolean expression 1 checking if User's company name = 'Aras'
- Boolean expression 2 seeing if the current item life cycle state is 'Released'
- Overall Condition requiring that both (AND) are satisfied

Each boolean expression is defined as an evaluation of Attributes and constants via Comparison Operators, which result in TRUE or FALSE based on equality, greater or less than, equal to or greater/less than, LIKE (%) and inequality. Comparison Operators are covered in detail in [Section 2.1.1.2](#), special helper functions in [Section 2.1.1.3](#).

MAC Boolean expressions support xClass and xProperties, refer to [Section 2.1.2](#).

As previously mentioned, MAC Policy typically evaluates attributes associated with a User (requestor) vs. attributes on the (requested) Item to decide whether (or not) to revoke that User's access to the Item. Therefore, MAC Conditions most often involve dual expressions of the general form:

<User attribute expression> AND <Item attribute expression>

User attributes are referenced as **CurrentUser**.<attribute>, and Item attributes are accessed as **CurrentItem**.<attribute>:

- **CurrentUser** is always the current logged in user in the expression being evaluated in the Rule.
- **CurrentItem** is always the item being accessed, as configured in the MAC Policy under the "Applied To" tab.

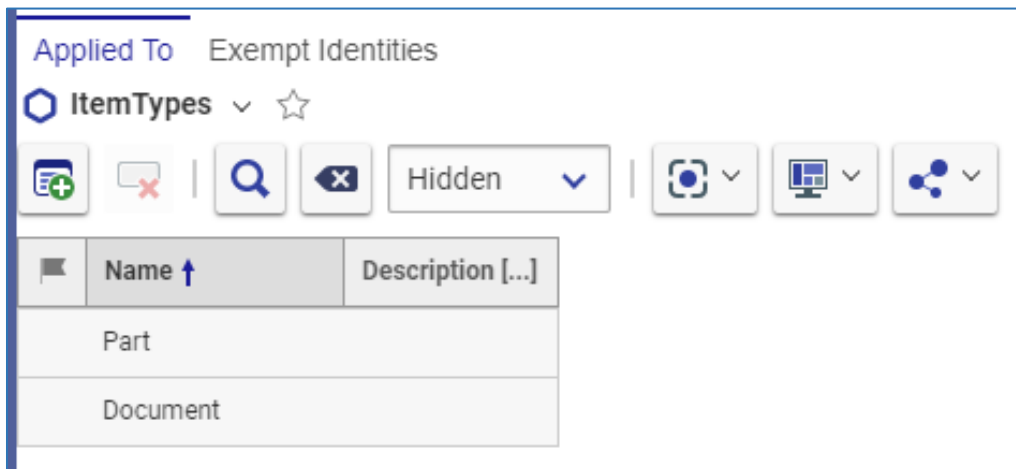


Figure 6.

Consider the example described in Table 2.

Table 1: The first example of a conditional Permission

Applied To	Access Rights	Condition
Documents, Parts	Get, Discover	CurrentUser .clearance >= CurrentItem .security_level_required

The User has *Get and Discover* access to all Documents and Parts based on existing Permissions. However, if this Policy Rule was activated, then any user whose clearance level was less than the level required by the Document or Part being requested would have access rights revoked for that specific item instance upon attempt to access.

MAC does not necessarily require User or Item duality, but conditions of singularity can almost always be achieved using simple Groups or Teams and standard Permissions instead; see Table 3.

Table 2: The second example of a conditional Permission

Applied To	Access Rights	Condition
Custom Secret ItemType	Get, Discover	CurrentUser .clearance > 3

The **Custom Secret** ItemType can simply have a Permission assigned that allows members of the **Clearance3** group to have the **Get** and **Discover** Access Rights. Users can be moved in or out of the group by administrators without adding and maintaining properties on user instances, etc.

In summary, with very few exceptions MAC Policies employ Rules based on Conditions that juxtapose attributes of the current User against attributes of the current Item.

1.1.5 MAC Attributes

In MAC the term *attribute* is a general abstraction for any value or set of values that can be used within a boolean expression. They may be single-valued, or multi-valued, and used interchangeably where type and cardinality allow.

MAC Attributes may any one of the following:

1. A Constant of the correct datatype, cardinality, and value.
5. A Property defined on the current ItemType being accessed.
6. A Property on the User ItemType for the current user making the access request.
7. Environment Attribute dynamically set by execution of a (custom) Method.
8. Derived Multi-valued Attribute produced from the results of a Query Definition.

1.1.6 Enabling Properties for Use in MAC Expressions

Within a *CurrentItem* expression only those Properties that are added to container item *mp_PolicyAccessItem* will become available for use in MAC expressions. If you need additional properties to be available in the *CurrentItem*.<attribute> list shown here then the property should be added to this Item. Specific details are provided in [Section 2.1.1.1](#).

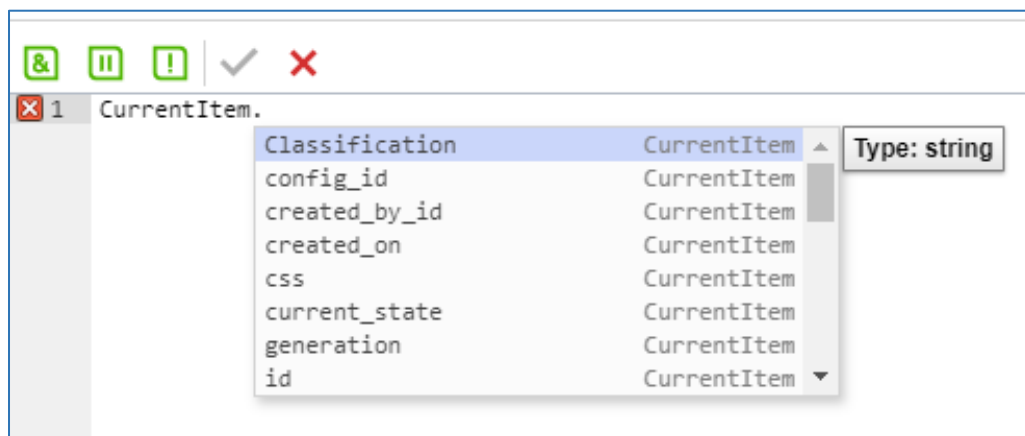


Figure 7.

CurrentUser attributes are available as they are configured in the *User* itemtype. Only administrators should be allowed to edit the User item to prevent potential access change due to modification of properties. See [Section 2.1.1](#) for details regarding this restriction.

1.1.7 Environment Attributes

Environment attributes provide a way to customize specific access control circumstances such as geographical location, or the time an access request is being made – essentially any calculated attribute value for use in expressions. Recall once again the simple example from the Introduction [Section 1.1](#):

- a) Current **Item** (E.g., Document) has a property **requires security** set to **TRUE**.
- b) AND: The current **User** has a property **security clearance** with a value greater than 0.
- c) AND: The current **User** property **foreign national** is set to **FALSE**.
- d) AND: The **Environment Attribute Within_Accessible_Hours** is set to **TRUE**.

Within_Accessible_Hours (d) does not exist as a Property anywhere, it is a calculated ‘virtual’ attribute whose value is generated by execution of a Method.

MAC provides the **Environment Attribute** ItemType under **Administration** --> **Access Control** in the TOC with the UI shown in figure 8.

The screenshot shows a web-based form for configuring an 'Environment Attribute'. The form has a title bar 'Environment Att...' and a main title 'Environment Attribute 1'. Below the title are three buttons: 'Save', 'Done', and 'Delete'. The form is divided into sections by a horizontal line. The first section is labeled 'Environment Attribute' and contains four fields: 'Name' (a text input), 'Description' (a larger text area), 'Type' (a dropdown menu currently showing 'Boolean'), and 'Get Value Method' (a dropdown menu with a three-dot icon). The form is styled with a light blue and white color scheme.

Figure 8.

Note that a “Get Value Method” is required. It is the responsibility of the administrator to define and provide a Method as described in [Section 2.2](#).

Once this is completed, the named attribute becomes available in MAC Condition expressions.

1.1.8 Derived Multivalued Attributes

As of Innovator 12.0 SP6, access administrators may take advantage of a new capability in MAC to use the results of a Query Definition to generate a *derived multivalued* attribute for use in their MAC policy’s boolean expressions.

In the Query Definition shown in the following diagram, *Document* is the root item for the query definition, and *Part Document* reference (relationship) defines a path to a *Part* ItemType parent. When this query executes, all of the “Restriction Level” target property values from all parent Parts are derived into a multivalued attribute as a collection of integer values [1, 0, 3]. This collection can be referenced in boolean expressions by name.

Note: Any time spaces are used in an attribute name use [square brackets] in the expression to refer to it.

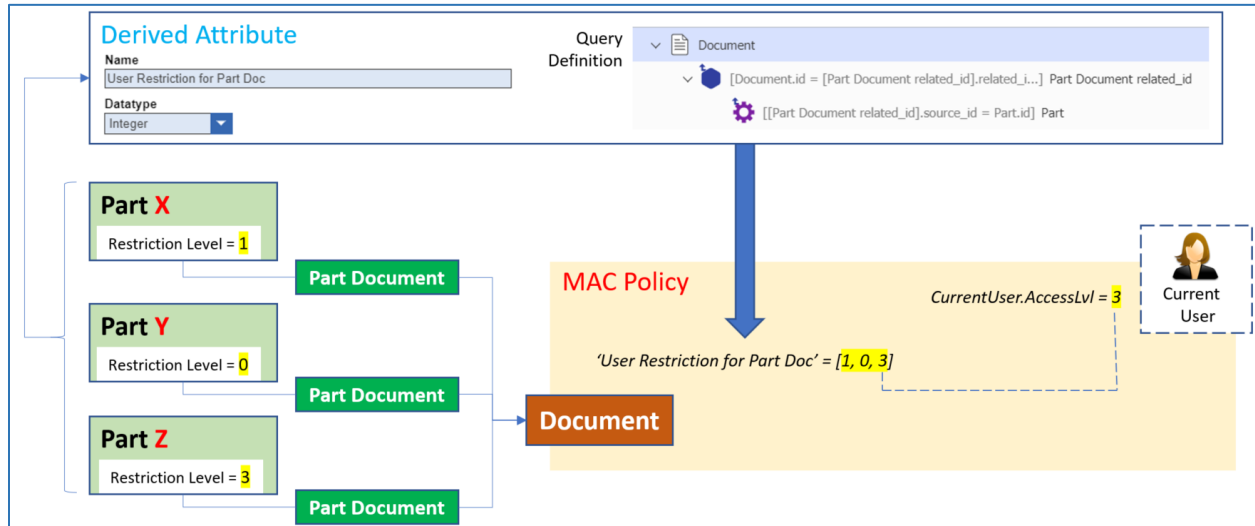


Figure 9.

For the sake of simplicity *CurrentUser.AccessLvl* is a single valued attribute i.e., an integer property called 'AccessLvl' on the User Itemtype in the example above. Alternatively, a derived multi-valued attribute or a constant value could be used on the User Item as needed. In any case, these attributes can be used in a MAC Rule's condition expression as follows:

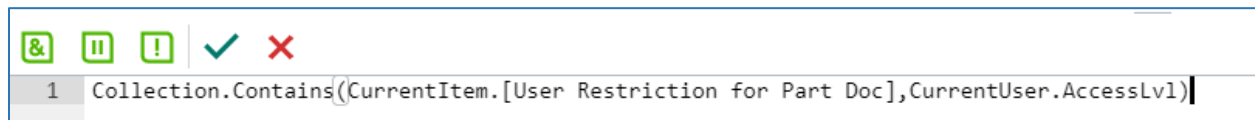


Figure 10.

The above Condition would only be true if the derived attribute collection for the current item *contains* the access level value from the current user, otherwise it is false.

1.1.8.1 Derived Attributes vs. Single-valued Attributes

Derived attributes provide greater flexibility over single-valued attributes for a number of reasons. It is not necessary to have the attribute configured as a Property on the *CurrentUser* or *CurrentItem* if the property can be 'derived' from a related item whether a parent or child relationship. Thus, it is not required to modify the User (Core) Itemtype if related to the parent item containing the attribute.

As discussed previously, Derived attributes are multi-valued. MAC boolean expressions can evaluate *multivalued* derived attributes as 'Collections' using set-based operators (Contains, Overlaps, IsEmpty).

Note: Property-based (single-valued) Attributes may result in better performance vs. Derived Attributes, which will provide greater flexibility. This should be taken into consideration when designing and testing a MAC Policy implementation.

2 Establishing MAC Policies

Aras Innovator MAC Policies are used to control user access to Items through a set of MAC Policy Rules. MAC Policy Rules control access rights for Get, Update, Delete, Can Discover, and Show Permission Warning to the set of ItemTypes which the MAC Policy is applied to.

2.1 Creating a New MAC Policy

You can find MAC Policies in Aras Innovator by clicking **Administration --> Access Control --> MAC Policies** in the TOC. The following menu appears:

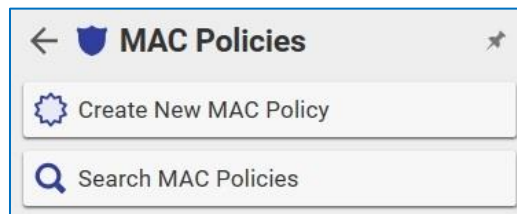


Figure 11.

Only users with Administrative permissions have the ability to create MAC Policies. Once you create a MAC Policy, you must specify the Name and save it. Once you do that, you can create MAC Policy rules. Use the following procedure:

2. Click **Create New MAC Policy**. The following screen appears:

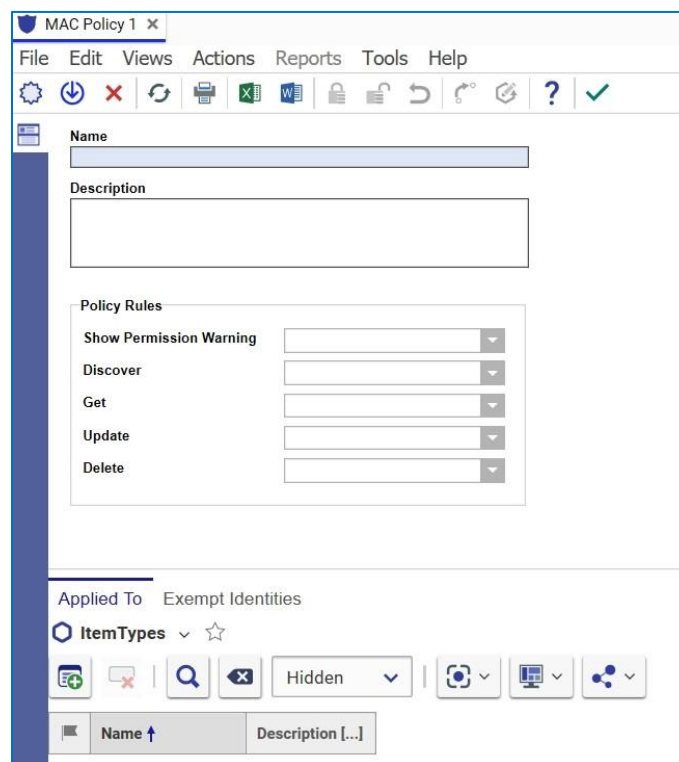


Figure 12.

9. Enter the Policy Name in the **Name** field and click . The MAC Policy Editor icon appears in the left margin.

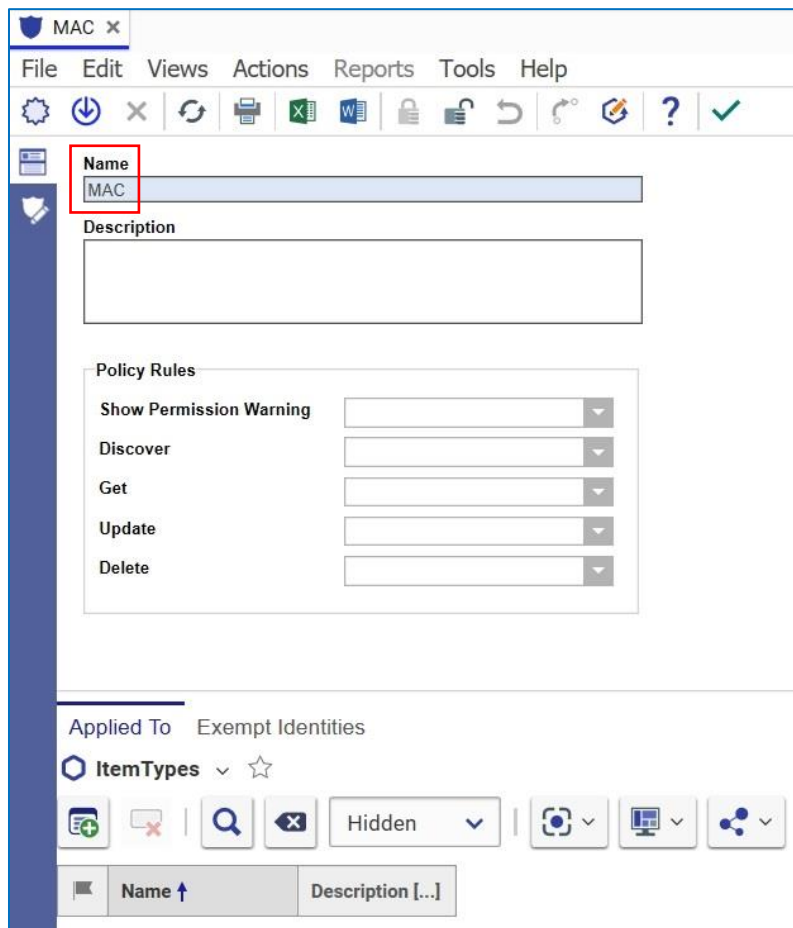


Figure 13.

2.1.1 Creating a MAC Condition



To create a new Condition, switch to the MAC Policy Editor view by clicking on the MAC Policy Editor icon.



Selecting the **New Condition** icon makes the lower portion of the window available to enter the Condition statement for the Policy Rule:

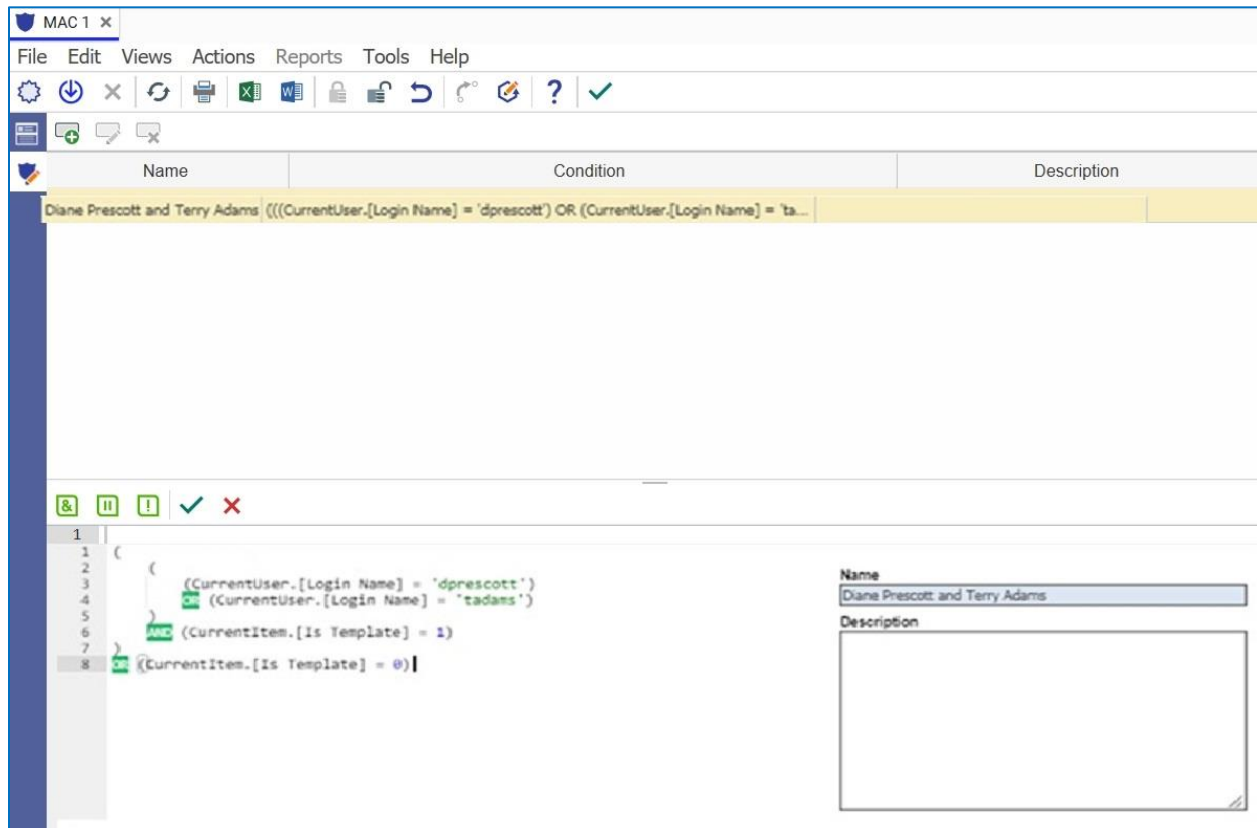


Figure 14.

The access administrator can create N Conditions within any given policy. The Condition editing is done in the lower pane, and the upper frame displays saved Conditions previously created within this MAC Policy Item.

A MAC Condition is comprised of one or more Boolean expressions combined by Logical Operators AND, OR, and NOT. Boolean expressions can reference CurrentItem, CurrentUser attributes which may be:

- Property-based attribute obtained from the Item being accessed—CurrentItem.<attribute_name>.
- Property-based attribute from the User making access request—CurrentUser.<attribute_name>.
- Environment Attribute—dynamically generated on invocation via Method execution.
- Derived Multivalued Attribute—created using the Derived Attribute Definition item.
- xClass or xProperty reference used as an attribute.

Expressions may also use constant values (hard-coded values).

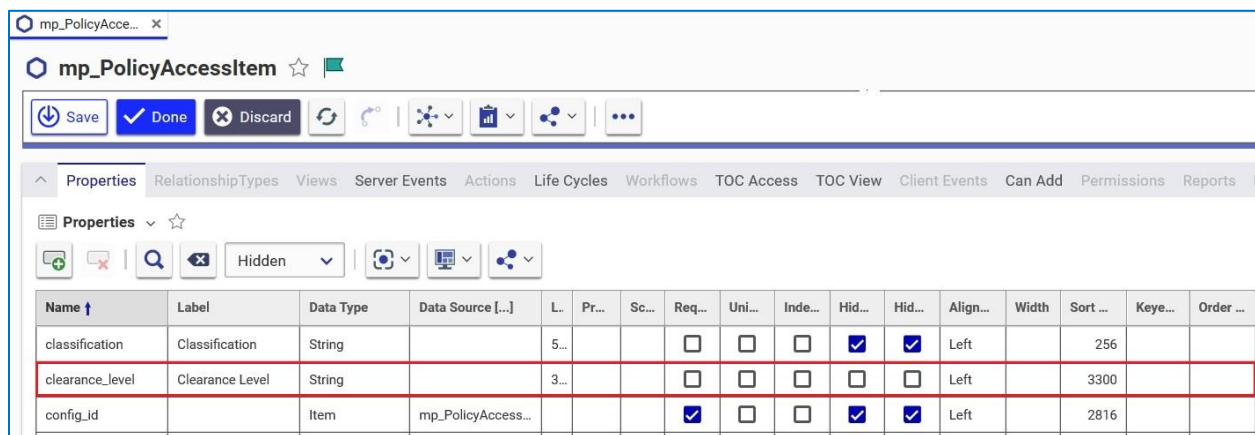
The following syntax rules apply to Policy Rule Condition statements:

- Values are case sensitive.
- String literals text must be enclosed between quotation marks ('text'), quotation escapes with back slash (\).
- A Constant can act as an operand when you use it in a comparison. Otherwise, a Constant is a string type.
- Operators and precedence are the same as those used in SQL languages.
- Operators are not case sensitive.
- Arithmetic operators are not supported.
- Parentheses can be used to override operator precedence.
- Comparing two strings follows Transact-SQL rules.

Note: If the Policy Rule uses a Property on the User ItemType and a user can edit their own User Item, then the user is capable of changing their level of access to any of the ItemTypes that a MAC Policy is applied to. This can also apply if the Edit control is not restricted in a MAC Policy and other Permissions are based on editable Properties.

2.1.1.1 Adding Supported ItemType Properties

Only properties that are attached to the **mp_PolicyAccessItem** ItemType can be referenced in a Policy Rule Condition statement. To add additional properties, open the **PolicyAccessItem** ItemType and add properties that will need to be referenced by the MAC Policy Rules.



Name ↑	Label	Data Type	Data Source [...]	L...	Pr...	Sc...	Req...	Uni...	Inde...	Hid...	Hid...	Align...	Width	Sort ...	Keye...	Order ...
classification	Classification	String		5...			☐	☐	☐	☒	☒	Left		256		
clearance_level	Clearance Level	String		3...			☐	☐	☐	☐	☐	Left		3300		
config_id		Item	mp_PolicyAccess...				☒	☐	☐	☒	☒	Left		2816		

Figure 15.

Properties added to the PolicyAccessItem must exactly match the properties of the ItemTypes that the MAC Policies are being applied to. All ItemTypes that the MAC Policy is being applied to must have the property, which is being referenced, otherwise validation will fail when the Admin attempts to save the Policy. The following data types are supported:

- String
- Integer
- Float
- Decimal
- Boolean
- Date

- Item
- List

In the previous example, the **clearance_level** property has been added and thus a Policy Rule can reference the property using the following syntax: **CurrentItem.[Clearance Level]**.

2.1.1.2 Supported Comparison Operators for Boolean Expressions

Table 4 lists operators that you can in the Condition statement of a Policy Rule.

Table 3: Supported comparison operators for Boolean expressions

Operation	Name	Usage	Meaning
=	Equals	valueRef1 = valueRef2	TRUE if left value is equal to right value.
>	Greater Than	valueRef1 > valueRef2	TRUE if left value is greater than right value.
<	Less Than	valueRef1 < valueRef2	TRUE if left value is less than right value.
>=	Greater Than or Equal To	valueRef1 >= valueRef2	TRUE if left value is greater than or equal to right value.
<=	Less Than or Equal To	valueRef1 <= valueRef2	TRUE if left value is less than or equal to right value.
!=	Not Equal To	valueRef1 != valueRef2	TRUE if left value is not equal to right value.
LIKE	Like	valueRef1 LIKE valueRef2	True is left value matches the right value (pattern). The syntax for the “LIKE” operator is exactly the same as in Transact-SQL.

2.1.1.3 Available Helper Methods

Table 5 lists available methods that you can use within the Condition statement of a Policy Rule.

Table 4: Available helper methods

Method	Response
CurrentUser.IsMemberOf(<Identity Name>)	Returns true if the current user is a member of a (non-system) Identity <Identity Name>, otherwise it returns false.
CurrentUser.IsMemberOf(Property<Item>)	Returns true if the current user is a member of the Item Property of type Identity. For example: CurrentUser.IsMemberOf(CurrentItem.identity_id)

Method	Response
CurrentUser.IsMemberOf(<multival attribute>)	Returns true if the current user is a member of at least one of (non-system) Identities from Collection <multival attribute>
String.Contains(<StringToSearch>, <SearchForString>)	Returns true if <SearchForString> is a substring of <StringToSearch>, otherwise it returns false.
CurrentItem.HasUserVisibilityPolicyAccess()	Returns true if the current user has access to the current item based on the active User Visibility Rules. This function can only be applied to User, Alias, Identity Item Types.

2.1.2 Using xClasses and xProperties in MAC Policy Conditions

You can use xProperties that are associated with the mp_PolicyAccessItem ItemType as item attributes in MAC Policy conditions. You should specify supported xProperties in the Allowed xProperties relationship tab in the mp_PolicyAccessItem. You can also use xProperties that are associated with the User ItemType as user attributes. Using xProperties in MAC Policy conditions may cause some performance penalties but it has the advantage of being able to specify access control for an item xProperty independently from access control for the item itself (and therefore all the item regular properties).

The rules for calculating a MAC Policy condition that is using an xProperty in cases when the xProperty is undefined follow those for AML. You can explicitly check in a MAC Policy condition to see if an xProperty is defined on an item or a user using the built-in functions described in section [2.1.2.1](#).

Warning DO NOT edit xProperty definitions that are being used in active MAC policies. Doing so results in a system failure. To adjust xProperty definitions, you must first disable the MAC policy. When you finish updating the xProperty definition, re-enable the MAC policy.

In MAC Policy conditions you can also check if an item or a user is classified by an xClass using the built-in functions described in section [2.1.2.1](#).

2.1.2.1 Using Methods to Verify Item Classification

Table 6 lists methods that enable you to check in MAC policy conditions to see if an item or user is classified using a particular xClass or xProperty.

Table 5: Methods for Item classification verification

Method	Response
CurrentItem.IsXPropertyDefined(<xPropertyName>)	Returns true only when the <xPropertyName> is defined on CurrentItem.
CurrentItem.IsClassifiedByXClass(<xClassName>)	Returns true only when the CurrentItem is classified by <xClassName>.

Method	Response
CurrentUser.IsXPropertyDefined(<xPropertyName>)	Returns true only when <xPropertyName> is defined on Current.User.
CurrentUser.IsClassifiedByXClass(<xClassName>)	Returns true only when CurrentUser is classified by <xClassName>.

For more information about xClasses and xProperties, refer to the Extended Classification guide.

2.2 Defining Environment Attributes

Environment attributes grant a user certain access rights to an Item based on specific circumstances such as the geographical location or the time an access request is made. For example, if the user makes the request outside of work hours, the request is denied. If the same user makes the same request during work hours, their request is accepted.

To create Environment Attributes in Aras Innovator:

1. Click **Administration --> Access Control --> Environment Attributes** in the TOC. The menu shown in figure 16 appears.



Figure 16.

2. Click **Create New Environment Attribute**. A blank **Environment Attribute** dialog appears.

Figure 17.

3. Enter the appropriate information in the **Name**, **Description**, **Type**, and **Get Value Method** fields.

The screenshot shows the 'work_hours' Environment Attribute configuration page. At the top, there is a tab labeled 'work_hours' with a close button. Below the tab, there is a header area with a location pin icon, the text 'work_hours', a star icon, and a flag icon. A toolbar contains buttons for 'Save', 'Done', 'Discard', and several icons for undo, redo, and other actions. The main content area is titled 'Environment Attribute' and contains four fields: 'Name' with the value 'work_hours', 'Description' with the text 'Returns True if the server time is between 8AM and 8PM, otherwise returns false.', 'Type' with a dropdown menu set to 'Boolean', and 'Get Value Method' with a dropdown menu set to 'isWorkHours'.

Figure 18.

4. Click  to save and unclaim the attribute.

Each attribute has a unique Name, Type, and custom method (specified in the “Get Value Method” property) that is called by Aras Innovator to get the attribute value for each request to Aras Innovator. The supported data types for Environment Attributes are Boolean, String, and Integer. The environment attribute values are considered when calculating MAC Policy conditions for determining if an access request should be granted if the conditions use the environment security attribute. An environment security attribute with the name <attr_name> is referenced in a MAC Condition as ‘\$<attr_name>’.

See Section [3.1](#) for an example of a method that can be specified in the Get Value Method field.

Note: It is the responsibility of the system administrator/implementation specialist to create custom server methods that return the environment security attribute non-NULL values to Aras Innovator. The custom methods should be created using the EnvironmentAttributeMethod method template.

2.3 Derived (Multi-Valued) Attributes

These special attributes are configured independently (like Environment Attributes) and become available in the Condition Editor ([Section 2.1.1](#)) when successfully configured. Derived Attribute Definitions reside under **Administration** --> **Access Control** in the TOC.

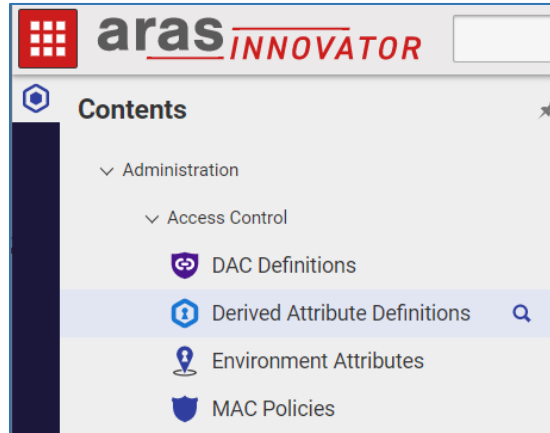


Figure 19.

Derived Attributes use an embedded Query Definition to specify the Path from the Root (CurrentItem) to a related Leaf Item and its Target Property from which the Derived Attribute values are extracted.

Refer to the following section for an explanation of Derived Attributes.

2.3.1 Configuring a Derived Attribute

A Derived Attribute has the structure shown in Figure 20 and described in Table.

The screenshot shows the 'Derived Attribute Definition' configuration form. At the top is a toolbar with buttons: Save, Done, Discard, and several icons for undo, redo, and other actions. The form has two main sections: 'Derived Attribute Definition' and 'Attribute Queries'.

In the 'Derived Attribute Definition' section, there are three fields:

- Name** (1): A text field containing 'User Restriction for Part Doc'.
- Datatype** (2): A dropdown menu set to 'Integer'.
- Description** (3): A text area containing 'Conditional Access Given Restriction Level of Parent Part -> Document'.

The 'Attribute Queries' section is below. It has a toolbar with icons for adding, deleting, and searching queries, and a 'Hidden' dropdown. Below the toolbar is a table with the following data:

Applied To [...]	Leaf Item	Target Property
Document	Part	restriction_level

The 'restriction_level' property in the table is highlighted with a yellow box and labeled with a '4'.

Figure 20.

Table 6: The Derived Attribute structure

1	Name	Symbol by which this derived attribute is referenced (Required). Enclose spaces in [square brackets] when used in an expression.
2	Datatype	Datatype of values contained in a derived attribute collection (Required). Must match Target Property type, and vice-versa.
3	Description	Text description of Attribute for reference.
4	Attribute Query	A key Relationshiptype that defines how to derive values for this attribute:
<i>First-time Entry Only</i>	=> “Applied To” Itemtype	The Itemtype that will define the scope of “CurrentItem” in Boolean expressions; also, the Root Item of the Path defined by the <i>Embedded Query Definition</i> . This field is a Required initial entry and becomes read-only once the Path is defined.
<i>Embedded (Not visible)</i>	=> (Query Definition)	Internally stored query definition describing (a) the Path from Root item to Leaf item and (b) Target Property on the Leaf item to derive into the collection (see diagram below).
<i>R/O</i>	=> Leaf Item	The Item at the endpoint of the Query Path. Target Properties are derived from this item. There can only be one Leaf Item in the query path.
<i>R/O</i>	=> Target Property	Name of the Property on the Leaf Item used to derive Attribute values. Type must match Datatype [2].

The **Query Definition** is accessed by *double-clicking the **Leaf Item** column** of the Attribute Query. The Root item is pre-populated as the **Applied To** Itemtype. Using the standard QD user interface, define the Path from Root to Leaf Item, and also select the Target Property. On completion, the read-only **Leaf Item** and **Target Property** columns will be populated.

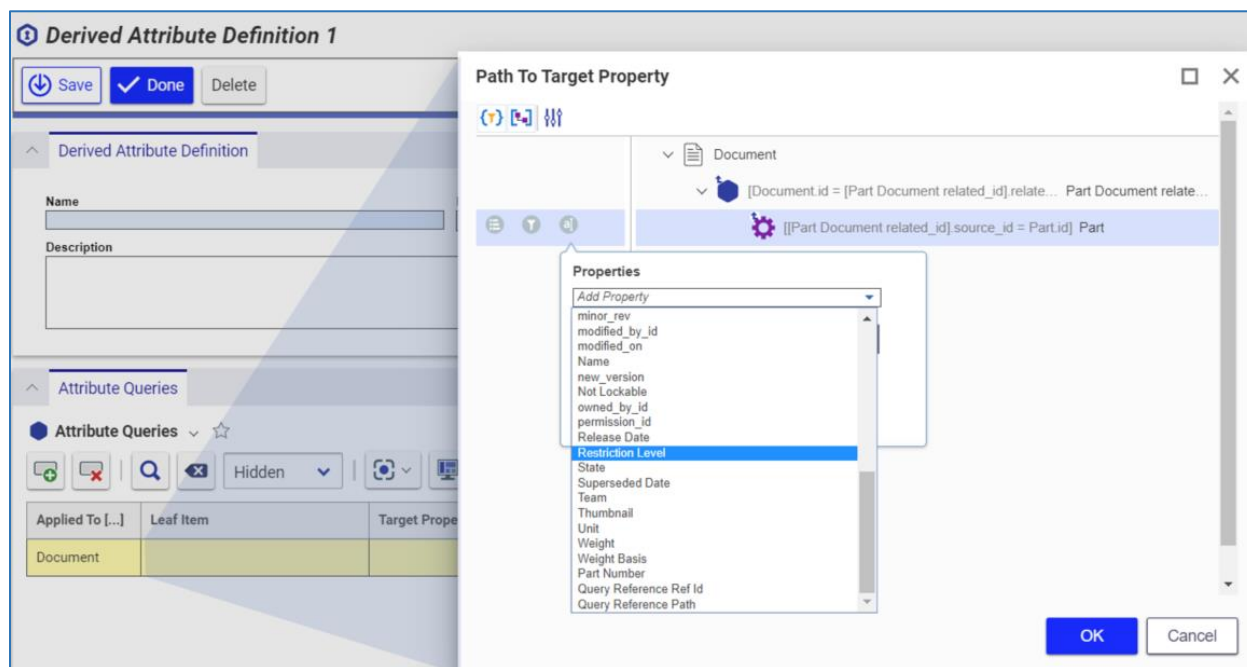


Figure 21.

2.3.1.1 Using a derived attribute in a Boolean expression

Derived Multivalued Attributes are Collections, and the operators described in Table 8 support their use in Condition logic (Boolean expressions).

Table 7: Collection operators

Operator	Description
<code>Collection.IsEmpty(<multival attribute>)</code>	TRUE if Collection is empty
<code>Collection.Contains(<multival attribute>, value)</code>	TRUE if Collection <multival attribute> contains the value <value> (type value)
<code>Collection.Overlaps(<multival attribute1>, <multival attribute2>)</code>	TRUE if Collection <multival attribute1> has values in common with <multival attribute2>

2.4 Applying Policy Rules

Once you create Conditions, you may assign them to specific Access Rights. There are five drop down lists:

- **Show Permission Warning** – determines if, when permissions are restricted, the user receives a standard permission warning.
- **Discover** – determines if the user can view an Item within a search grid.
- **Get** – determines if the user can open Forms and view all the Properties of an Item.
- **Update** – determines if a user can claim an Item, make changes, and save those changes.
- **Delete** – determines if a user can delete an Item.

Selecting a Condition from the dropdown list applies that Condition to the access right. The user is denied access if the selected conditions are not met.

Figure 22.

Note: If a Rule is not selected for an access right, then the MAC Policy will not apply any restrictions to that access right.

Standard Aras Innovator role-based permissions and all applicable MAC Policy rules must grant access to an Item before a user is able to access it.

2.4.1 Applying MAC Policies

Once you establish the Policy Rules, you must assign the MAC Policy to the desired ItemTypes and then activate the MAC policy. Assigning the MAC policy to ItemTypes is done by adding them to the Applied To Relationship.

To activate a MAC Policy, unclaim the policy and select the 'Activate' Action. When Activating a MAC Policy, all users except for the admin activating the MAC Policy should be logged off and prevented from accessing the system until the MAC Policy is activated.

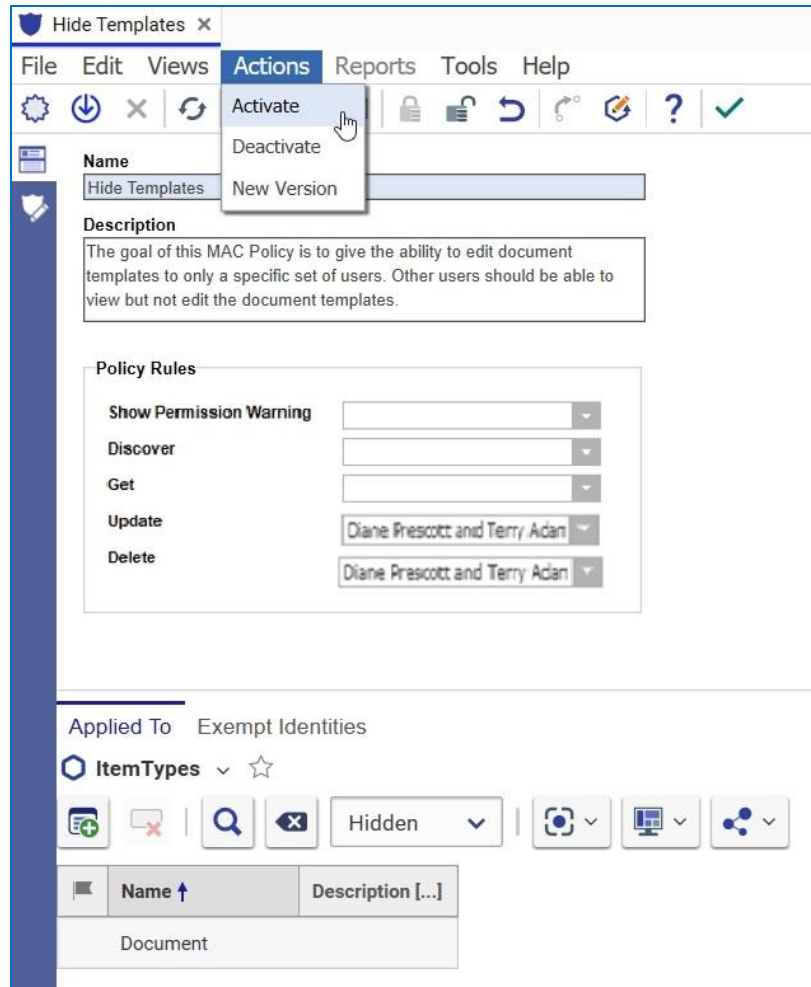


Figure 23.

2.4.2 Updating MAC Policies

Once activated, a MAC Policy can be deactivated or versioned. The **Deactivate** action sets a MAC Policy to the **Inactive** state. When a MAC Policy is **Inactive**, the Policy Rules are not applied to control access.

To modify a previously active MAC Policy, the Policy must be versioned through the **New Version** Action. The previous revision of the MAC Policy is promoted to the **Archived** state only after the new version of the Policy is activated.

2.5 Exempt Identities

Each MAC Policy includes a list of "Exempt identities." If the user making an access request has an identity that is included in the "Exempt identities" list, MAC Policy rules are not applicable to the request.

3 Appendix

3.1 Sample Environment Attribute Method

The following example shows how to write a method that sets the value of the attribute to true from 8AM to 8PM server time, Monday through Friday:

```
//MethodName=CSharp:Aras.Server.Core.AccessControl.EnvironmentAttributeMethod;
var startWorkTime = new TimeSpan(8, 0, 0);
var endWorkTime = new TimeSpan(20, 0, 0);
var currentDateTime = DateTime.Now;
var isWorkDay = DayOfWeek.Monday <= currentDateTime.DayOfWeek &&
currentDateTime.DayOfWeek <= DayOfWeek.Friday;
var isWorkTime = startWorkTime <= currentDateTime.TimeOfDay &&
currentDateTime.TimeOfDay <= endWorkTime;
var isWorkHours = isWorkDay && isWorkTime;
attribute.SetValue(isWorkHours);
```

Note: It is recommended to make the Methods OS agnostic to use them with different operating systems. The main incompatibility issues in operating systems that need to be considered when implementing the Methods are related to file path case-sensitivity, path separators, OS-specific line-endings, OS-specific code. For more information about cross-platform development please see section "2.3 Cross-platform development" in "Aras Innovator 33 - Programmer's Guide" document.

3.2 Sample MAC Policy Rule

The example shown in Figure 24 is a Policy Rule that restricts Update rights for certain Parts.

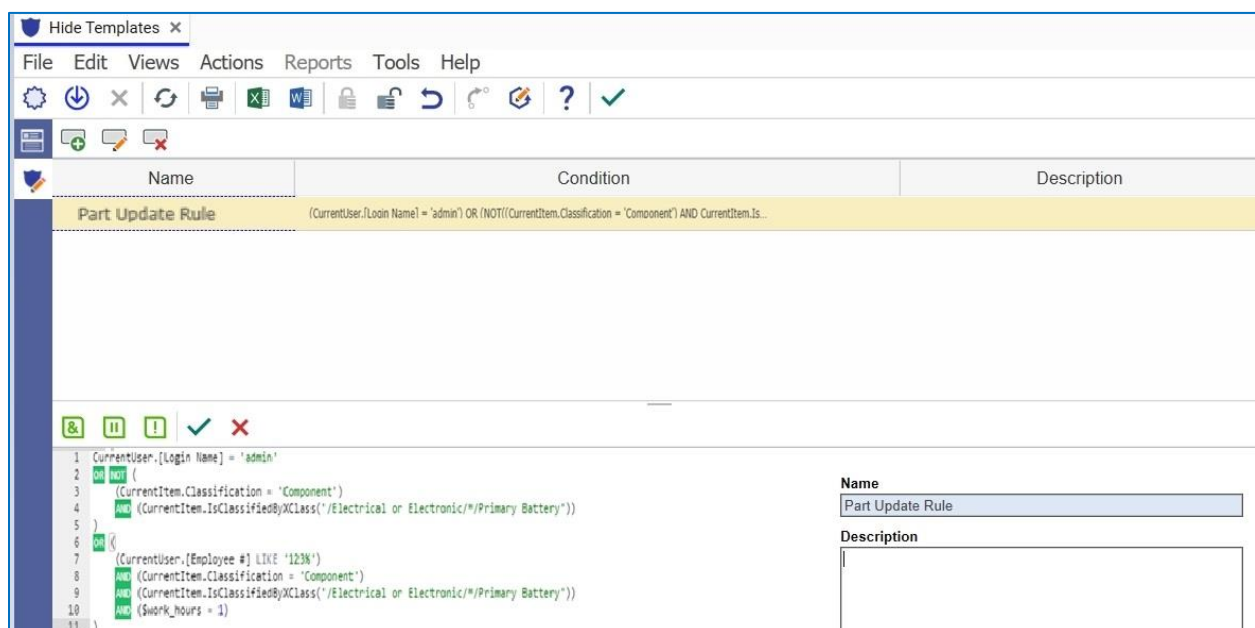


Figure 24.

The policy rule specifies a few conditions:

1. The Admin can update any part.
2. Any user with Standard Aras Innovator Permissions can update parts as long as they do not have both the 'Component' Classification and the 'Primary Battery' xClass.
3. Users with an Employee number beginning with '123' can Update Parts during work hours which have both the 'Component' Classification and the 'Primary Battery' xClass.